

Residual Non-Convergence of p^*_S After the Soft-Threshold Fix: Three Secondary Mechanisms

Technical Note — Segmented Search Model Solver

April 2026

Abstract

After the soft-threshold fix documented in the previous note eliminated the grid-snapping discontinuity in the skilled inner loop, the skilled-block outer loop still fails to converge on $p^*_S(x)$ for a substantial fraction of SMM parameter draws, even as θ_S converges cleanly. This note identifies three secondary mechanisms that were suppressing the joint-state Anderson(m=1) acceleration, and documents targeted fixes for each: (A) pre-damping of the proposals that are fed to Anderson, (B) the post-hoc $p^{\text{oj}} \leftarrow \max(p^*_S, p^{\text{oj}})$ projection, and (C) the hard $\max(\cdot, 0)$ clamp on raw surpluses S^0, S^1 inside the inner loop. Two complementary changes are also applied: per-block scaling of the global-loop Anderson on $[U_S; m_S]$, and a McQueen–Porteus acceleration in the unskilled inner loop for symmetry with the skilled side. All changes are confined to `skilled.jl`, `unskilled.jl`, and `solver.jl`.

Contents

1	Recap: what the soft-threshold fix solved	2
2	Symptom	2
3	Why scalar-θ Anderson is <i>not</i> the right answer	2
4	Issue A: Anderson is extrapolating the damped map	2
4.1	The bug	2
4.2	Fix A	3
5	Issue B: the $p^{\text{oj}} \leftarrow \max(p^*, p^{\text{oj}})$ projection creates a hidden period-2	4
5.1	The bug	4
5.2	Fix B: reparameterize on the gap	4
6	Issue C: hard $\max(\cdot, 0)$ on inner-loop surpluses	5
6.1	The residual kink	5
6.2	Why Anderson struggles with C^0 -but-not- C^1 maps	5
6.3	Fix C: smooth the clamp	6
7	Global-loop scaling	6
7.1	Motivation	6
7.2	Fix: locked per-block scales	7
7.3	On Anderson memory at the global layer	7
8	Unskilled inner loop: McQueen–Porteus for symmetry	7

9	Summary of changes	8
10	Verification checklist	8
11	What this note does <i>not</i> change	9

1 Recap: what the soft-threshold fix solved

The previous note identified a period-2 limit cycle in $p^*_S(x)$ driven by a discontinuous fixed-point map: the inner loop used `pcut_index` to hard-zero surpluses below the cutoff, making the surplus surface (and hence the updated cutoff) a step function of p^*_S . Two fixes were applied:

- (i) **Soft thresholding.** The hard zero was replaced with a piecewise-linear weight $\omega_j \in [0, 1]$ on the straddling cell, making the surplus surfaces continuous in p^*_S .
- (ii) **Joint Anderson(m=1).** The acceleration was extended from θ_S alone to the full state $[\theta_S; p^*_S; p^{oj}]$.

These fixes resolved the dominant grid-snapping cycle but exposed three smaller obstructions that had been masked by the larger one. Those obstructions are the subject of this note.

2 Symptom

In SMM batches run after the soft-threshold fix, many candidate parameter draws produce a trace of the form:

Iteration	$\Delta\theta_S$	$\ \Delta p^*_S\ _\infty$	$\ \Delta p^{oj}\ _\infty$
n	$< 10^{-9}$	$\sim 3 \times 10^{-3}$	$\sim 3 \times 10^{-3}$
$n+1$	$< 10^{-9}$	$\sim 2 \times 10^{-3}$	$\sim 2 \times 10^{-3}$
$n+2$	$< 10^{-9}$	$\sim 3 \times 10^{-3}$	$\sim 3 \times 10^{-3}$
$n+3$	$< 10^{-9}$	$\sim 2 \times 10^{-3}$	$\sim 2 \times 10^{-3}$

The amplitude is 2–3 orders of magnitude smaller than the raw pre-fix cycle (~ 0.4 – 0.9), so the hard-threshold issue is no longer the culprit. The near-periodicity of the residual and the wide gap between $\Delta\theta_S$ (converged) and $\|\Delta p^*_S\|_\infty$ (non-converged) together point to an Anderson acceleration that is *seeing* a period-2 signal it cannot extrapolate through.

3 Why scalar- θ Anderson is *not* the right answer

Before moving to diagnoses, one question from the maintainer deserves a direct answer: *should the skilled block revert to Anderson on θ_S alone*, as in the unskilled block?

No. The original period-2 documented in the previous note lived in p^*_S , not θ_S ; indeed θ_S converged cleanly even then because the aggregation over worker types x averaged out the per-type grid-snapping discontinuities. Restricting Anderson to θ_S would send us back to the pre-fix regime: the p^*_S period-2 would re-emerge on top of a happily converged θ_S , and `converged_S = false` would return immediately on the restored full convergence criterion. The joint-state design is correct; what we will change is the *signal* Anderson sees (Fix A) and the *state* it operates on (Fix B).

4 Issue A: Anderson is extrapolating the damped map

4.1 The bug

Prior to this note, step (C) of the skilled outer loop damped the proposals *before* the stationary solve and *before* the Anderson call:

```
# (C) Stationary distribution (using proposed cutoffs temporarily)
damp = sim.damp_pstar_S
for ix in 1:Nx
```

```

    sc.pstar[ix] = clamp01(damp * pst_prop[ix] + (1-damp) *
        pstar_old[ix])
    sc.poj[ix]    = max(sc.pstar[ix],
        clamp01(damp * poj_prop[ix] + (1-damp) *
            poj_old[ix]))
end
solve_stationary_skilled!(model; mS_in = mS_in)
theta_raw = update_theta_skilled(model)

# (E) Anderson on [theta; pstar; poj]
x_old = vcat([theta_old], pstar_old, poj_old)
f_raw = vcat([theta_raw], sc.pstar, sc.poj) # <-- already damped
x_new = anderson1_update!(aa_joint, x_old, f_raw)

```

The p_S^* , p^{oj} components of $\mathbf{f_raw}$ are not $T(p_S^{\text{old}}, p^{oj\text{old}})$ — they are $\delta \cdot T(\cdot) + (1 - \delta) \cdot \text{old}$, where $\delta = \text{damp_pstar_S}$. Define the raw residual $r = f - x$ and the damped one $\tilde{r} = \tilde{f} - x$. A short calculation gives

$$\tilde{r} = \delta \cdot r, \quad (1)$$

so the Anderson(m=1) update

$$\alpha = - \frac{\langle r_0, r_1 - r_0 \rangle}{\|r_1 - r_0\|^2} \quad (2)$$

receives a signal that has been uniformly scaled down by δ on the p_S^* and p^{oj} components. Two concrete consequences follow.

Consequence A1: period-2 signal is suppressed

In a genuine period-2 cycle $f_k \approx x_{k-1}$, Anderson(m=1) detects the reversal via the vector $r_1 - r_0$ and chooses $\alpha \approx 0.5$. When the residual is scaled by δ , the two cycle states move closer together by a factor of δ , so $r_1 - r_0$ is likewise compressed. With default $\delta \approx 0.3\text{--}0.5$, $\|r_1 - r_0\|^2$ can fall below the `denom < 1e-12` guard in `anderson1_update!`, at which point the routine returns $\mathbf{f}$ unchanged — i.e. pure damped Picard — which is exactly the iteration known to be non-convergent here.

Consequence A2: θ is inconsistent with the cutoffs

The θ component of $\mathbf{f_raw}$ is computed from `solve_stationary_skilled!`, which was run with the *damped* p_S^* , p^{oj} installed in `sc`. But the p_S^* , p^{oj} components of $\mathbf{f_raw}$ are also the damped values. So Anderson sees:

$$f_{\text{raw}} = (\theta_{\text{raw}}^{\text{damp}}, p^{\text{damp}}, p^{oj\text{damp}}; p^{\text{damp}}, p^{oj\text{damp}}),$$

and tries to extrapolate a fixed point of the map whose image this is. The “true” map Anderson ought to be accelerating is the one generated by the raw proposals; instead, it is accelerating a δ -dependent composition. This misalignment matters when δ is not close to 1.

4.2 Fix A

Install the raw proposals directly when Anderson is on, and solve the stationary distribution from those:

```

# (C) Install cutoffs for stationary solve
if sim.use_anderson
    for ix in 1:Nx
        sc.pstar[ix] = pst_prop[ix]
    end
end

```

```

        sc.poj[ix]    = max(pst_prop[ix], poj_prop[ix])
    end
else
    # Damped Picard safety net for the use_anderson=false branch
    damp = sim.damp_pstar_S
    for ix in 1:Nx
        sc.pstar[ix] = clamp01(damp * pst_prop[ix] + (1-damp) *
                                pstar_old[ix])
        sc.poj[ix]   = max(sc.pstar[ix],
                            clamp01(damp * poj_prop[ix] + (1-damp) *
                                    poj_old[ix]))
    end
end
end
solve_stationary_skilled!(model; mS_in = mS_in)
theta_raw = update_theta_skilled(model)

```

With raw proposals installed, θ_{raw} is consistent with p_S^{prop} and p^{ojprop} , and Anderson sees the undistorted residual $r = f - x$. Damping is preserved as a Picard fallback when Anderson is disabled, so the non-Anderson code path behaves identically to before.

5 Issue B: the $p^{\text{oj}} \leftarrow \max(p^*, p^{\text{oj}})$ projection creates a hidden period-2

5.1 The bug

The original post-Anderson block enforced feasibility with a point-wise max:

```

for ix in 1:Nx
    sc.pstar[ix] = clamp01(x_new[1 + ix])
    sc.poj[ix]   = clamp01(x_new[1 + Nx + ix])
    sc.poj[ix]   = max(sc.pstar[ix], sc.poj[ix])    # enforce poj >=
    pstar
end

```

For worker types x where the extrapolated p^{oj} lands slightly below the extrapolated p_S^* , this projection snaps p^{oj} up to p_S^* . On the *next* iteration, the cutoff-finding step (B) re-separates them (since $p^{\text{ojprop}} > p^{\text{prop}}$ is generically true for active markets). Anderson then extrapolates again, the projection binds again, and a fresh period-2 orbit is created. Critically, this cycle is *invisible* to Anderson because it happens *after* the Anderson update — when `anderson1.update!` computes $r_1 - r_0$ on the next call, both x_k and f_k already reflect the post-projection state, so the residual sequence looks smooth.

5.2 Fix B: reparameterize on the gap

Instead of accelerating (p_S^*, p^{oj}) and then projecting, we accelerate (p_S^*, gap) where $\text{gap} = p^{\text{oj}} - p_S^* \geq 0$:

$$\mathbf{z} = \begin{pmatrix} \theta_S \\ p_S^*(x_1) \\ \vdots \\ p_S^*(x_{N_x}) \\ \text{gap}(x_1) \\ \vdots \\ \text{gap}(x_{N_x}) \end{pmatrix} \in \mathbb{R}^{1+2N_x}. \quad (3)$$

```

if sim.use_anderson
    gap_old = poj_old - pstar_old      # >= 0 by invariant
    gap_prop = poj_prop - pst_prop     # >= 0 by step (B)
    x_old = vcat([theta_old], pstar_old, gap_old)
    f_raw = vcat([theta_raw], pst_prop, gap_prop)
    x_new = anderson1_update!(aa_joint, x_old, f_raw)

    sc.theta = max(x_new[1], 1e-14)
    for ix in 1:Nx
        pstar_new = clamp01(x_new[1 + ix])
        gap_new = clamp(x_new[1 + Nx + ix], 0.0, 1.0)
        sc.pstar[ix] = pstar_new
        sc.poj[ix] = clamp01(pstar_new + gap_new)
    end
end
end

```

The constraint $p^{\text{oj}} \geq p^*_S$ is now folded into the state representation: clamping $\text{gap} \geq 0$ is a feasibility projection on a state component Anderson itself can observe on the next call. Post-Anderson, `sc.poj` is reconstructed as `clamp01(pstar + gap)`, which automatically satisfies both $p^{\text{oj}} \geq p^*_S$ and $p^{\text{oj}} \in [0, 1]$. No post-hoc `max` projection is needed.

6 Issue C: hard $\max(\cdot, 0)$ on inner-loop surpluses

6.1 The residual kink

Inside the skilled inner loop, the raw surpluses S^0, S^1 were clamped to be non-negative before being multiplied by ω_j :

```

S0 = max((PS_x * x * pj - (r + nu) * U_new + lam * I) / base, 0.0)
S1 = max((PS_x * x * pj - (r + nu) * U_new - sigma + lam * I +
          f * beta * tail_Emax_j) / (base + f * tail_mass_j), 0.0)

```

Consider the contribution $J^0(x, p_j) = (1 - \beta) \omega_j(p^*) S^0(x, p_j)$ as a function of p^* , for a fixed grid node p_j :

- For p^* well above p_j (fully inactive): $\omega_j = 0$, so $J^0 = 0$ trivially.
- For p^* in the straddling cell (p_{j-1}, p_j) : $\omega_j < 1$, but S^0 evaluated at p_j is still positive, so $J^0 > 0$ and varies smoothly with p^* via ω_j .
- For p^* just above p_j (straddling cell (p_j, p_{j+1})): $\omega_j \in (0, 1)$, but raw S^0 at p_j is *negative* because $p_j < p^*$. The clamp sets $S^0 \leftarrow 0$, so $J^0 = 0$.

As p^* descends across p_j , $J^0(x, p_j)$ jumps from 0 (straddling from above, S^0 clamped) to a positive value with non-zero slope (straddling from below). This is a C^0 -but-not- C^1 kink at *every* grid node p_j . The kink appears in the tail integral $I(x)$ and hence in $U_S(x)$, which feeds back to the next iteration's p_S^{prop} via the cutoff-finding step (B).

6.2 Why Anderson struggles with C^0 -but-not- C^1 maps

Anderson(m=1) is built on a local linearization assumption: $r_k \approx J(x_k - x^*)$ for some Jacobian J of the fixed-point map at x^* . A piecewise-smooth map with kinks violates this: J effectively depends on which “side” of the nearest kink x_k falls on, and the “ $r_1 - r_0$ ” vector conflates two different linearizations. In practice, Anderson does not fail completely — it still makes progress

— but its effective contraction rate degrades, and for parameter draws where the fixed point sits within $O(\Delta p)$ of a grid node, the iteration can stall at the kink.

6.3 Fix C: smooth the clamp

Define a C^∞ replacement for $\max(\cdot, 0)$:

$$\text{smooth_pos}(x, \varepsilon) := \frac{1}{2}(x + \sqrt{x^2 + \varepsilon^2}), \quad \varepsilon = 10^{-8}. \quad (4)$$

Properties:

- Smoothness: C^∞ everywhere.
- Consistency with max: $\text{smooth_pos}(x, \varepsilon) \rightarrow \max(x, 0)$ as $\varepsilon \rightarrow 0$ (pointwise, and uniformly outside any neighbourhood of 0).
- Bandwidth: the transition region where smooth_pos differs from $\max(x, 0)$ by more than ε is $|x| \lesssim 10\varepsilon$, i.e. 10^{-7} with $\varepsilon = 10^{-8}$. This is below `tol_outer_S` = 10^{-7} , so the smoothing cannot bias the converged fixed point beyond the solver’s own tolerance.
- Non-negativity: $\text{smooth_pos}(x, \varepsilon) \geq 0$ for all x ; equals $\varepsilon/2$ at $x = 0$. The economic interpretation “surplus is non-negative” is preserved.

The two call sites in `skilled_inner_loop!` become:

```
S0 = smooth_pos((PS_x * x * pj - (r + nu) * U_new + lam * I) / base)
S1 = smooth_pos((PS_x * x * pj - (r + nu) * U_new - sigma + lam * I +
                f * beta * tail_Emax_j) / (base + f * tail_mass_j))
```

One other $\max(\dots, 0.0)$ survives in `solve_stationary_skilled_x!` (on the employment density $e(x, p_j)$); this was left untouched because (i) it bites rarely in practice, and (ii) it enters the feedback loop only through the normalisation in `update_theta_skilled`, which already has its own θ -specific Anderson smoothing. If diagnostics later indicate it matters, the same `smooth_pos` substitution would apply.

7 Global-loop scaling

7.1 Motivation

The global fixed-point solver in `solver.jl` applies `Anderson(m=1)` to the joint link vector $[U_S; m_S]$. The two blocks live on different natural magnitudes:

- $U_S(x)$: a lifetime value, monetary-unit, $O(1)$ – $O(10)$.
- $m_S(x) = (\varphi/\nu)t(x)$: a training inflow density, often $O(0.01)$ – $O(0.1)$ depending on φ/ν .

Anderson’s residual norm $\|r\|^2$ is then dominated by whichever block is larger; the coefficient α Anderson computes is chosen primarily to tame *that* block, and the smaller block rides along without being accelerated. This is a scale problem, not a smoothness problem, but it can produce slow convergence when the period-2 mode happens to live in the small-magnitude block.

7.2 Fix: locked per-block scales

Introduce scales s_U and s_M that rescale each block before Anderson sees it:

$$\tilde{\mathbf{x}} = (U_S/s_U; m_S/s_M), \quad \tilde{\mathbf{f}} = (U_S^{\text{raw}}/s_U; m_S^{\text{raw}}/s_M), \quad (5)$$

feed $(\tilde{\mathbf{x}}, \tilde{\mathbf{f}})$ to `anderson1_update!`, then unscale block-wise on the way out.

The scales are *locked* at iteration 1 rather than recomputed every iteration. Two reasons:

- Anderson’s internal state (`aa.x_prev`, `aa.f_prev`) stores the scaled values from the previous call. If the scales change between iterations, the stored r_0 is on a different scale than the current r_1 , and the α formula becomes meaningless.
- `Anderson(m=1)` returns f unchanged on its first call regardless of scale, so locking at $it = 1$ is equivalent to locking at $it = 0$ with a known-good value.

```
# at top of solve_model!
s_U = 1.0
s_M = 1.0

# inside the loop, after step B
if it == 1
    s_U = max(maximum(abs, US_raw), 1.0)
    s_M = max(maximum(abs, mS_raw), 1.0)
end

x_vec = vcat(US_old ./ s_U, mS_old ./ s_M)
f_vec = vcat(US_raw ./ s_U, mS_raw ./ s_M)
x_new_s = sim.use_anderson ? anderson1_update!(aa, x_vec, f_vec) :
    f_vec
US_new = x_new_s[1:Nx] .* s_U
mS_new = max.(x_new_s[Nx+1:end] .* s_M, 0.0)
```

A floor of 1.0 prevents amplifying numerical noise when a block is genuinely near zero (in which case the scale becomes a no-op and reproduces the previous unscaled behaviour).

7.3 On Anderson memory at the global layer

The maintainer specifically noted that in this model, the skilled/unskilled feedback produces an oscillation that `Anderson(m=1)` is already well-suited to break, so bumping memory to $m = 2$ or $m = 3$ is not needed. This matches the theoretical expectation: `Anderson(m=1)` is optimal for breaking period-2 cycles, which is exactly the dominant failure mode here. Memory has not been changed.

8 Unskilled inner loop: McQueen–Porteus for symmetry

The skilled inner loop already uses McQueen–Porteus acceleration on U_S (the span of signed Bellman increments gives an estimate of the contraction rate ρ , which is turned into a uniform shift on U_S). The unskilled inner loop, prior to this note, iterated on U^{search}, U, T with no such acceleration.

Because T is pinned to U_S^{in} in one step and $U = \max(U^{\text{search}}, -c(x) + T)$ inherits from U^{search} on searching types, the correct target for McQueen–Porteus is U^{search} . The implementation mirrors the skilled side: track signed increments δU^{search} across x , compute the span, estimate ρ , apply the shift. The convergence test uses raw (pre-shift) increments, matching `skilled_inner_loop!`. See `unskilled.jl` lines 137–180.

Caveat. On worker types where the training branch $U = -c(x) + T$ binds, a shift on U^{search} does not translate into motion on U , so the acceleration is effectively inactive for those types. Near the fixed point the shift is small, so this is harmless; far from the fixed point it means the acceleration is weaker on types near the training margin, which is the correct conservative behaviour.

9 Summary of changes

All changes are confined to `skilled.jl`, `unskilled.jl`, and `solver.jl`. Data structures, parameter files, and the `Anderson1` primitive itself are unchanged.

Component	Before	After
Skilled outer, step (C)	Damped cutoffs installed before stationary solve; Anderson sees damped residual $\delta \cdot r$	Raw cutoffs installed when Anderson is on; damping kept as Picard fallback when Anderson is off
Skilled outer, step (E) Anderson state	$[\theta; p_S^*; p^{\text{oj}}]$ with post-hoc $p^{\text{oj}} \leftarrow \max(p^*, p^{\text{oj}})$	$[\theta; p_S^*; \text{gap}]$ with $\text{gap} \in [0, 1]$ clamped and $p^{\text{oj}} = \text{clamp01}(p^* + \text{gap})$
Skilled inner loop surplus	$S^0 = \max(\cdot, 0), S^1 = \max(\cdot, 0)$	$S^0 = \text{smooth_pos}(\cdot), S^1 = \text{smooth_pos}(\cdot)$ with $\varepsilon = 10^{-8}$
Global loop Anderson	$[U_S; m_S]$ unscaled	$[U_S/s_U; m_S/s_M]$ with scales locked at $it = 1$, floor 1.0
Unskilled inner loop	Plain Picard on U^{search}	McQueen–Porteus shift on U^{search} with raw-residual convergence test

10 Verification checklist

Before rolling the fixes into an SMM batch, it is worth running a small number of targeted diagnostics:

- 1. Smoothing bias.** Re-solve a baseline candidate that converged cleanly under the pre-fix code. The converged $(\theta_S, p_S^*, p^{\text{oj}}, \theta_U, p_U^*)$ should agree with the baseline to within $\sim \varepsilon = 10^{-8}$. If the discrepancy is larger, `smooth_pos`’s ε is biasing the fixed point; drop it to 10^{-10} .
- 2. Anderson signal.** Instrument `anderson1_update!` to log the chosen α and the denominator $\|r_1 - r_0\|^2$ per call. Under the pre-fix code, the denominator often fell below the 10^{-12} guard on SMM candidates with non-converged p_S^* ; under the fix it should stay well above.
- 3. Projection-binding rate.** Under the pre-fix code, count the fraction of `ix` per outer-loop iteration where $p^{\text{oj}\text{prop}} > p_S^{\text{prop}}$ but post-Anderson $p^{\text{oj}} < p_S^* + \epsilon$. Under the fix, this is structurally 0 (the `max` projection no longer exists).
- 4. Convergence rate.** On a suite of SMM candidates, compare the number of outer-loop iterations to convergence. A modest reduction is expected from Fix A alone (no damping on the Anderson path); the larger wins come from fewer `converged.S = false` returns.
- 5. Global-layer speedup.** Compare global iteration counts with and without per-block scaling. The benefit is candidate-specific: negligible when U_S and m_S happen to be on similar scales, larger (often 20–30% fewer iterations) when they differ by orders of magnitude.

11 What this note does *not* change

For completeness, a few items that were considered and rejected:

- *Anderson memory at the global layer.* Kept at $m = 1$, per the maintainer's request and the argument in Section 7.
- *McQueen–Porteus at the global layer.* The span/ρ interpretation is tied to Bellman contractions; the global fixed point on $[U_S; m_S]$ is a general fixed point without that structure, so the acceleration does not transfer cleanly.
- *Reverting to scalar- θ Anderson on the skilled side.* Rejected for the reasons in Section 3.
- *The remaining $\text{max}(\dots, 0.0)$ in `solve_stationary_skilled_x!`.* Untouched pending diagnostic evidence it matters. If Test 4 above shows unexplained slowness traceable to the stationary solve, the same `smooth_pos` substitution would apply there.